

Improving ADC Resolution by Oversampling and Averaging

---

by

A. N. Author

A dissertation submitted in partial fulfilment
of the requirements for the degree of

Master of Engineering in Electronic Engineering

Supervisor: Dr A. Supervisor
Department: Department of Electronic Engineering

University of Example

May 2026

Declaration of Authorship

I, A. N. Author, declare that this dissertation, titled "Improving ADC Resolution by Oversampling and Averaging", and the work presented in it are my own. I confirm that:

- This work was done wholly while in candidature for a degree at University of Example.
- Where any part of this dissertation has previously been submitted for a degree or any other qualification at this or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this dissertation is entirely my own work.
- I have acknowledged all main sources of help.
- Where the dissertation is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: A. N. Author

Date: May 2026

Abstract

Low-cost microcontrollers usually include a 12-bit analogue-to-digital converter, which is not enough for sensors whose output changes by only a small fraction of the input range. Oversampling and averaging is a well-known way to obtain more resolution from the same converter, at the cost of a lower output rate, but it only works if the conversion errors differ from one sample to the next, and the amount of noise needed for that is often stated loosely.

This dissertation examines the technique by simulation. An ideal 12-bit converter was modelled with Gaussian noise added at its input, and the effective resolution after averaging was measured for oversampling ratios from 1 to 256, with both a sine wave and static inputs. With 0.5 LSB rms of noise the simulated resolution followed the theoretical gain of one bit for every factor of four in oversampling ratio to within 0.03 bits, reaching 15.0 effective bits at a ratio of 256. With no noise, averaging a static input gave no improvement at all, and with 0.25 LSB rms the improvement stopped at around 14 bits.

The results show that about half an LSB of noise is enough for the method to work as predicted up to a ratio of 256, and that adding more than this costs resolution. The model does not include converter non-linearity, which averaging cannot correct, so the figures are an upper limit for a real device.

Acknowledgements

I would like to thank my supervisor, Dr A. Supervisor, for the guidance given throughout this project, and in particular for the suggestion to test static inputs separately from the sine wave, which turned out to be where the most useful results came from.

Thanks also go to the staff of the Department of Electronic Engineering for their help in the early stages of the work, and to my family for their patience during the writing of it.

Contents

Declaration of Authorship I

Abstract II

Acknowledgements III

List of Figures VI

List of Tables VI

Nomenclature VII

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Aim and objectives . . . 1
- 1.3 Structure of the dissertation . . . 1

2 Background 3
- 2.1 Quantisation error . . . 3
- 2.2 Oversampling and averaging . . . 3
- 2.3 The need for noise . . . 4

3 Method 5
- 3.1 Approach . . . 5
- 3.2 The model . . . 5
- 3.3 Measuring the resolution . . . 6
- 3.4 Test 1: sine wave input . . . 6
- 3.5 Test 2: static inputs . . . 6

4 Results 7
- 4.1 Test 1: sine wave input . . . 7
- 4.2 Test 2: static inputs . . . 7

5 Discussion 9
- 5.1 Agreement with theory . . . 9
- 5.2 Why a small amount of noise is not enough . . . 9
- 5.3 The cost of too much noise . . . 9
- 5.4 Practical implications . . . 9
- 5.5 Limitations . . . 10

6 Conclusion **11**
6.1 Further work 11

References **12**

A Simulation Source Code **13**

B Results of the Static Test **16**

List of Figures

1 Oversampling and averaging. The converter takes M readings for each output sample, and the output is their mean. 4
2 Effective resolution against oversampling ratio for a sine wave input with 0.5 LSB rms of noise. 7
3 Error in the averaged result for static inputs, at four levels of input noise. . . . 8

List of Tables

1 Parameters of the simulation. 5
2 Effective resolution for a sine wave input with 0.5 LSB rms of noise, compared with the prediction of eq. (4). 7
3 Error in the averaged result for static inputs, in LSB rms, at each level of input noise. 16

Nomenclature

σ_n Root-mean-square noise at the converter input

M Oversampling ratio, as used in equations

N Resolution of the converter, in bits

q Size of one quantisation step

ADC Analogue-to-Digital Converter

ENOB Effective number of bits

LSB Least significant bit, the smallest step the converter can resolve

OSR Oversampling ratio, the number of conversions averaged into each output sample

SNR Signal-to-noise ratio

1 Introduction

1.1 Motivation

MOST low-cost microcontrollers include an analogue-to-digital converter (ADC) with a resolution of 12 bits. With a 3.3 V reference this gives a step size of 0.8 mV, which is adequate for many measurements, but not for sensors whose output moves by only a small part of the input range. A load cell or a thermocouple, for example, may need steps of tens of microvolts to give a useful reading.

The usual answer is to add an external converter with a higher resolution, which adds cost, board area and another device to configure. An alternative is to take many readings with the converter already present and average them. This is known as oversampling, and in principle it gives one extra bit of resolution for every factor of four in the number of readings averaged [1]. It costs nothing in hardware, but the output rate falls by the same factor, so it suits signals that change slowly.

The method has a condition attached, which is that the error in each reading must differ from the last. If every reading of a steady input returns the same code, their average is that same code and nothing has been gained. Some noise at the input is therefore needed, and application notes generally say that it should be around one LSB, without showing what happens when there is less. That gap is what this project sets out to fill.

1.2 Aim and objectives

The aim of the project is to establish how much input noise an ideal 12-bit converter needs for oversampling and averaging to deliver the resolution that theory predicts. The objectives are:

1. To review the theory of quantisation noise and of oversampling, and to state the predicted resolution as an equation that can be tested.
2. To build a simulation model of an ideal converter followed by an averaging stage, and to check it against a case with a known answer.
3. To measure the effective resolution for oversampling ratios from 1 to 256, for a sine wave and for static inputs, at several noise levels.
4. To compare the results with the theory, and to state the conditions under which the two agree.

1.3 Structure of the dissertation

Section 2 reviews the theory and the published guidance. Section 3 describes the simulation model and the two tests carried out with it. The results are presented in section 4 and discussed

in section 5, along with the limitations of the model. Section 6 gives the conclusions and suggests further work. The source code of the simulation is listed in appendix A, and the full table of results for the static test is in appendix B.

2 Background

2.1 Quantisation error

An ADC with a resolution of N bits divides its input range into 2^N steps of size q, and reports the step that the input falls in. The difference between the true input and the centre of the reported step is the quantisation error, and for an ideal converter it lies between $-q/2$ and $+q/2$.

When the input moves across many steps between one sample and the next, the error is close to being uniformly distributed over that range and unrelated from sample to sample. Bennett showed that it can then be treated as noise added to the signal [2], with a root-mean-square value of

$$e_{\text{rms}} = \frac{q}{\sqrt{12}}. \tag{1}$$

Comparing this with the rms value of a full-scale sine wave gives the familiar expression for the signal-to-noise ratio of an ideal converter [3],

$$\text{SNR} = 6.02N + 1.76 \text{ dB}, \tag{2}$$

which is 74 dB for 12 bits. A real converter does a little worse than this, and its performance is often quoted by working eq. (2) backwards from the measured ratio to give an effective number of bits (ENOB).

2.2 Oversampling and averaging

If M readings are taken in the time that one was needed, and their errors are unrelated to each other, the error in their average is smaller than the error in a single reading by a factor of $\sqrt{M}$. This is the ordinary result for the mean of independent measurements, and it applies to quantisation error and to any other noise present, provided both are unrelated from one sample to the next [4]. If the noise at the converter input has an rms value of σ_n, the error in the averaged output is therefore

$$e_{\text{out}} = \sqrt{\frac{q^2/12 + \sigma_n^2}{M}}. \tag{3}$$

The number M is the oversampling ratio (OSR). The arrangement is shown in fig. 1.

Expressing eq. (3) as an effective number of bits, by comparing it with the error of an ideal converter in eq. (1), gives

$$\text{ENOB} = N - \tfrac{1}{2}\log_2\!\left(1 + \frac{12\sigma_n^2}{q^2}\right) + \tfrac{1}{2}\log_2 M. \tag{4}$$

The last term is the gain from oversampling, and it is half a bit for each doubling of M, or

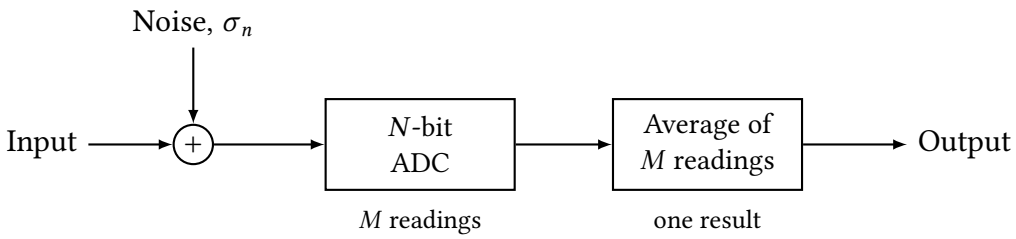


Figure 1: Oversampling and averaging. The converter takes M readings for each output sample, and the output is their mean.

one bit for each factor of four. The middle term is the price of the noise, as noise that helps the averaging also adds to the error that has to be averaged away. With $\sigma_n = q/2$ it is exactly one bit. Equation (4) is the prediction that the simulation in this project tests.

2.3 The need for noise

Equation (3) relies on the errors being unrelated from sample to sample. For a steady input and a noiseless converter this fails completely, as every reading returns the same code and the same error. Widrow and Kollár give the formal conditions under which quantisation error behaves as independent noise [5], and they are met when the input to the quantiser varies by a step or more.

The practical guidance follows from this. The application note for one widely used family of microcontrollers states that the signal must be oversampled by a factor of four for each additional bit, and that the method relies on noise of at least one LSB being present, which is to be added deliberately if the circuit does not provide it [1]. Added noise of this kind is known as dither. What the guidance does not show is how the result degrades as the noise falls below that level, or whether more noise than necessary does any harm, and those are the questions the tests in section 3 were designed to answer.

3 Method

3.1 Approach

The questions were investigated by simulation and not on hardware. On a real microcontroller the input noise cannot be set to a chosen value, and it is mixed with the converter's other imperfections, so the effect of the noise alone could not be separated out. A simulation allows the noise to be set exactly, including to zero, and the true input is known at every sample, so the error can be calculated directly. The cost of this choice is that the results describe an ideal converter, which is considered in section 5.

3.2 The model

The model has three parts, which follow fig. 1. The input is a voltage expressed in units of one LSB. Gaussian noise with a chosen rms value is added to it, and the sum is rounded to the nearest whole code and limited to the range of a 12-bit converter. Listing 1 shows this part. Each output sample is then the mean of M successive codes.

```
26 def adc(voltage_lsb, noise_rms):
27     """Ideal quantiser: add noise, round to the nearest code, clip."""
28     if noise_rms:
29         voltage_lsb += rng.gauss(0.0, noise_rms)
30     return min(max(math.floor(voltage_lsb + 0.5), 0), FULL_SCALE)
```

Listing 1: The model of an ideal converter with noise at its input.

The model was written in Python using only its standard library, so that it can be run without installing anything further, and the random number generator is given a fixed seed so that the results repeat exactly. The full source is listed in appendix A. The parameters are in table 1.

Parameter	Value
Converter resolution, N	12 bits
Oversampling ratios, M	1, 4, 16, 64, 256
Input noise, σ_n	0, 0.25, 0.5 and 1.0 LSB rms
Sine test: amplitude	2000 LSB, centred on mid-scale
Sine test: frequency	3.71 % of the output sample rate
Sine test: output samples per run	4096
Static test: input levels per run	2000, placed at random between two codes

Table 1: Parameters of the simulation.

3.3 Measuring the resolution

In both tests the error of each output sample is the difference between the averaged codes and the average of the true input over the same samples. The rms value of that error over the run, e_{out}, is converted to an effective number of bits by comparing it with eq. (1),

$$\text{ENOB} = N - \log_2\left(\frac{e_{\text{out}}\sqrt{12}}{q}\right), \tag{5}$$

so that an error equal to that of an ideal 12-bit converter gives exactly 12 bits.

3.4 Test 1: sine wave input

The first test uses a sine wave covering most of the input range, with 0.5 LSB rms of noise, and measures the resolution at each oversampling ratio. This is the case that eq. (4) describes, so it serves as the check on the model as well as the first result. The frequency was chosen so that the run does not contain a whole number of cycles, which avoids the same few codes being exercised repeatedly.

3.5 Test 2: static inputs

The second test holds the input steady for the whole of each average, which is the condition under which the method is expected to fail without noise. A single level would not be representative, as an input that happens to sit at the centre of a step has no error to begin with, so each run uses 2000 levels spread at random across one step and takes the rms error over all of them. The test was repeated at each of the four noise levels in table 1.

4 Results

4.1 Test 1: sine wave input

The results of the sine wave test are given in table 2 and plotted in fig. 2. The table is built from the file written by the simulation, so the figures in it are exactly those the model produced.

Oversampling ratio	Theory (bits)	Simulated (bits)	Error (LSB rms)
1	11.00	10.97	0.589
4	12.00	11.99	0.290
16	13.00	13.02	0.143
64	14.00	14.00	0.072
256	15.00	15.00	0.036

Table 2: Effective resolution for a sine wave input with 0.5 LSB rms of noise, compared with the prediction of eq. (4).

With no averaging the model gave 10.97 bits against a predicted 11.00, which confirms that the noise and the quantisation error combine as eq. (3) assumes. The resolution then rose by one bit for each factor of four in oversampling ratio, reaching 15.00 bits at a ratio of 256. The largest difference from the prediction at any ratio was 0.03 bits.

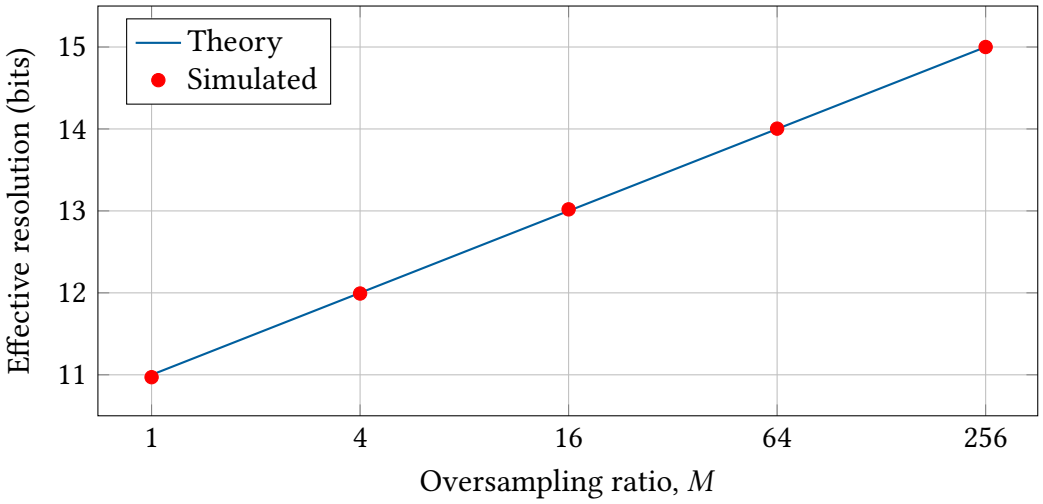


Figure 2: Effective resolution against oversampling ratio for a sine wave input with 0.5 LSB rms of noise.

4.2 Test 2: static inputs

Figure 3 shows the rms error for static inputs at each of the four noise levels, with the full figures in appendix B. The three cases behave quite differently.

With no noise the error stayed between 0.28 and 0.29 LSB at every ratio. This is the value of $q/\sqrt{12}$ from eq. (1), which means averaging 256 readings gave the same result as taking

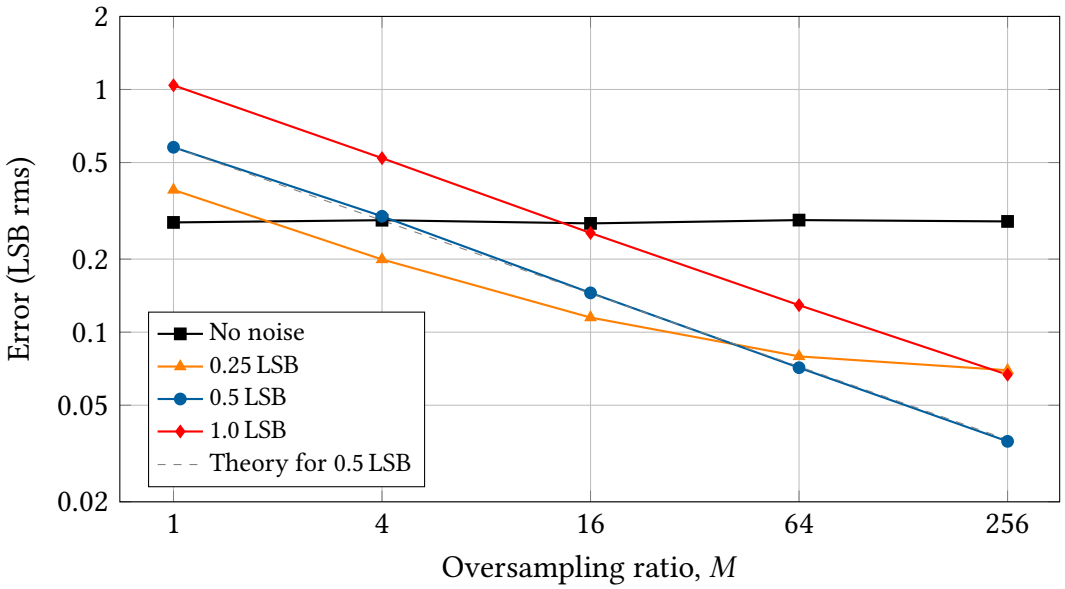


Figure 3: Error in the averaged result for static inputs, at four levels of input noise.

one.

With 0.5 LSB and 1.0 LSB of noise the error fell in proportion to $1/\sqrt{M}$ across the whole range, following eq. (3). At a ratio of 256 the errors were 0.036 LSB and 0.067 LSB, which correspond to 15.0 and 14.1 effective bits.

With 0.25 LSB of noise the error fell at first, from 0.39 LSB to 0.20 LSB at a ratio of 4, but then levelled out. At a ratio of 256 it was 0.070 LSB, about three times the 0.024 LSB that eq. (3) predicts, and little better than at a ratio of 64.

5 Discussion

5.1 Agreement with theory

The sine wave test agreed with eq. (4) to within 0.03 bits at every oversampling ratio, which is within what would be expected from a run of 4096 samples. This supports both the theory and the model, and it shows that for a signal which is itself moving across many codes the method works as the textbooks describe.

The static test is the more informative of the two, because it removes the help that a moving signal gives. Here the theory held only when there was enough noise. With 0.5 LSB rms or more, the static results matched the sine wave results, so at that level the noise alone is sufficient to make the errors independent and the nature of the signal no longer matters.

5.2 Why a small amount of noise is not enough

The result at 0.25 LSB can be explained by considering an input that sits a quarter of the way between two codes. With plenty of noise, the readings are spread over several codes in proportions that average out to the true value. With only a little, nearly all of the readings return the nearer code, and the occasional reading of the further one does not occur often enough to pull the average to the right place. The average then settles on a value that is wrong by a fixed amount, and because the amount is fixed, more averaging does not reduce it.

This is why the curve for 0.25 LSB in fig. 3 levels out and does not continue downwards. The averaging is still removing the random part of the error, but what remains is a systematic error that depends on where the input sits within the step.

5.3 The cost of too much noise

The results also show that noise beyond what is needed is not free. At a ratio of 256 the error with 1.0 LSB of noise was almost twice that with 0.5 LSB, which is a loss of nearly one bit, and to recover it the ratio would have to be raised by a factor of four. The middle term of eq. (4) predicts exactly this. The guidance to provide at least one LSB of noise is therefore safe, but it is not optimal, and a designer who can control the noise level would do better to aim for about half an LSB.

5.4 Practical implications

For a 12-bit converter with a 3.3 V reference, half an LSB is 0.4 mV rms. Many circuits will have this much noise without any being added, in which case the method can be used as it is. Whether a particular circuit does can be checked very simply, by holding the input steady

and looking at the spread of the raw codes. If nearly all of them are the same code, there is too little noise and dither will have to be added.

The other practical cost is time. An output rate of 100 Hz with a ratio of 256 needs the converter to run at 25.6 kHz, which is well within the reach of a typical microcontroller, but the same ratio at an output rate of 10 kHz would need 2.56 MHz, which is not.

5.5 Limitations

The model is of an ideal converter, and three of its simplifications affect how far the results can be carried over to a real one.

The first is that the steps of the model are all exactly the same size. A real converter has steps of slightly different sizes, described by its differential and integral non-linearity, and averaging does nothing to correct these, as they are the same on every reading. The 15 bits reached here should be read as an upper limit, with the real figure set by the linearity of the device in question.

The second is that the noise in the model is Gaussian and unrelated from one sample to the next. Noise picked up from a switching power supply or from the mains is neither, and noise that repeats in step with the sampling would not average away at all.

The third is that a plain average is a poor low-pass filter. It was used because it is what most firmware does, but a signal with content close to half the output rate would be attenuated by it, and a better decimation filter would be needed in that case [4].

6 Conclusion

The aim of this project was to establish how much input noise an ideal 12-bit converter needs for oversampling and averaging to deliver the resolution that theory predicts. Taking the objectives in turn:

1. The theory was reviewed and reduced to a single testable prediction, eq. (4), which gives the effective resolution in terms of the oversampling ratio and the input noise.

2. A simulation model was built and checked against the case with no averaging, where it gave 10.97 bits against a predicted 11.00.

3. The effective resolution was measured for oversampling ratios from 1 to 256, for a sine wave and for static inputs at four noise levels.

4. With 0.5 LSB rms of noise or more, the results agreed with the theory for both kinds of input, reaching 15.0 effective bits at a ratio of 256. With 0.25 LSB the improvement stopped at around 14 bits, and with no noise there was none for a static input.

The answer to the question is therefore that about half an LSB rms of noise is sufficient up to an oversampling ratio of 256, and that this is also close to the best choice, as doubling it cost almost a bit of resolution. This is a more specific figure than the one LSB or more given in the guidance that was reviewed.

6.1 Further work

The most useful next step would be to repeat the static test on a real microcontroller, using a precision voltage source for the input, to find how far the non-linearity of an actual converter limits what the simulation predicts.

The model could also be extended in two directions. The first is to replace the Gaussian noise with a deliberately added triangular waveform, which is simpler to generate in a circuit and may need a smaller amplitude for the same result. The second is to carry the oversampling ratio beyond 256, as the curve for 0.5 LSB must eventually level out in the same way as the curve for 0.25 LSB, and it would be useful to know where.

References

[1] Atmel Corporation, "AVR121: Enhancing ADC resolution by oversampling," Atmel Corporation, Application Note, 2005.

[2] W. R. Bennett, "Spectra of quantized signals," *Bell System Technical Journal*, vol. 27, no. 3, pp. 446–472, Jul. 1948.

[3] W. Kester, Ed., *The Data Conversion Handbook*. Oxford: Newnes, 2005.

[4] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Pearson, 2010.

[5] B. Widrow and I. Kollár, *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge: Cambridge University Press, 2008.

A Simulation Source Code

The complete source of the simulation is listed below. It uses only the Python standard library, and running it writes the two result files that the tables and figures in section 4 are built from.

```
"""Oversampling and averaging with an ideal N-bit ADC.

Produces the two result files used in the dissertation example:
  enob_vs_osr.csv   sine-wave test, effective bits against OSR
  dc_error.csv      static-input test, error at several noise levels

Standard library only. Run it with:  python oversampling_sim.py
"""
import csv
import math
import random
from pathlib import Path

N_BITS = 12
FULL_SCALE = 2 ** N_BITS - 1          # highest output code
OSR_VALUES = [1, 4, 16, 64, 256]
NOISE_LEVELS = [0.0, 0.25, 0.5, 1.0]  # input-referred noise, LSB rms
SINE_NOISE = 0.5                      # noise used for the sine-wave test
OUTPUT_SAMPLES = 4096                 # output samples per sine-wave run
DC_TRIALS = 2000                      # static levels tried per run
DATA_DIR = Path(__file__).resolve().parents[2] / "data"

rng = random.Random(20260614)         # fixed seed, so the results repeat


def adc(voltage_lsb, noise_rms):
    """Ideal quantiser: add noise, round to the nearest code, clip."""
    if noise_rms:
        voltage_lsb += rng.gauss(0.0, noise_rms)
    return min(max(math.floor(voltage_lsb + 0.5), 0), FULL_SCALE)


def enob(error_rms_lsb):
    """Effective bits, from the rms error against an ideal ADC's."""
    return N_BITS - math.log2(error_rms_lsb * math.sqrt(12.0))


def sine_test(osr, noise_rms):
    """Rms error of the averaged output for a sine wave input, in LSB."""
    amplitude, offset = 2000.0, FULL_SCALE / 2.0
    # The input frequency is 3.71 % of the output sample rate.
    step = 2.0 * math.pi * 0.0371 / osr
```

```
    total = 0.0
    for k in range(OUTPUT_SAMPLES):
        ideal = measured = 0.0
        for i in range(osr):
            v = offset + amplitude * math.sin(step * (k * osr + i))
            ideal += v
            measured += adc(v, noise_rms)
        total += ((measured - ideal) / osr) ** 2
    return math.sqrt(total / OUTPUT_SAMPLES)


def dc_test(osr, noise_rms):
    """Rms error of the averaged output for static inputs, in LSB."""
    total = 0.0
    for _ in range(DC_TRIALS):
        v = 1000.0 + rng.random()          # anywhere between two codes
        measured = sum(adc(v, noise_rms) for _ in range(osr)) / osr
        total += (measured - v) ** 2
    return math.sqrt(total / DC_TRIALS)


def write_csv(name, header, rows):
    """Write one result file, with Unix line endings."""
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    with open(DATA_DIR / name, "w", newline="") as f:
        out = csv.writer(f, lineterminator="\n")
        out.writerow(header)
        out.writerows(rows)


def main():
    rows = []
    for osr in OSR_VALUES:
        theory = enob(math.sqrt((1 / 12 + SINE_NOISE ** 2) / osr))
        err = sine_test(osr, SINE_NOISE)
        bits = [f"{theory:.3f}", f"{enob(err):.3f}"]
        rows.append([osr] + bits + [f"{err:.4f}"])
    write_csv("enob_vs_osr.csv", ["osr", "theory", "sim", "error"], rows)

    rows = []
    for osr in OSR_VALUES:
        errors = [dc_test(osr, n) for n in NOISE_LEVELS]
        rows.append([osr] + [f"{e:.4f}" for e in errors])
    header = ["osr"] + [f"n{int(n * 100):03d}" for n in NOISE_LEVELS]
    write_csv("dc_error.csv", header, rows)


```

```
90 if __name__ == "__main__":
91     main()
```

Listing 2: The simulation, `oversampling_sim.py`.

B Results of the Static Test

Table 3 gives the figures plotted in fig. 3. Each is the rms error, in LSB, over 2000 static input levels.

Oversampling ratio	No noise	0.25 LSB	0.5 LSB	1.0 LSB
1	0.283	0.386	0.578	1.040
4	0.289	0.199	0.300	0.522
16	0.281	0.115	0.145	0.256
64	0.290	0.080	0.071	0.129
256	0.286	0.070	0.036	0.067

Table 3: Error in the averaged result for static inputs, in LSB rms, at each level of input noise.